분야 입학연수 구분 개발 입문 학습시간 40시간 필수/선택 필수

# 컴퓨터에게 명령 내리는 말(파이썬) 처음 배우기

### 문제기술

기술적 설명




## 1. 미션 소개

프로그래밍 언어를 배우는 가장 확실한 방법은, 동작하는 프로그램을 처음부터 끝까지 직접 만들어 보는 것입니다.

문법을 아는 것과 그 문법으로 하나의 프로그램을 완성하는 것은 전혀 다른 경험입니다.

이 미션에서는 그 차이를 직접 확인합니다.

Python과 Git은 "내가 만든 프로그램이 왜 이렇게 동작하는지"를 설명하고, 그 과정을 기록하며 성장하기 위한 핵심 도구입니다.

이 미션에서는 터미널에서 동작하는 나만의 퀴즈 게임을 처음부터 끝까지 구현합니다.

Python 기본 문법을 사용해 입력/출력 흐름을 만들고, 클래스(객체 지향)로 코드를 역할별로 구조화합니다.

아울러 JSON 파일 저장을 통해 프로그램을 종료해도 퀴즈와 점수가 유지되도록 "데이터 영속성"을 경험합니다.

또한, Git으로 변경 이력을 관리합니다.

기능 단위로 커밋하고, 브랜치를 나눠 작업한 뒤 병합하며, GitHub에 저장소를 공개합니다.

이 경험은 이후 팀 협업(브랜치 전략, PR, 코드 리뷰)으로 자연스럽게 확장됩니다.

## 2. 최종 결과물

다음 3가지 조건을 만족하는 Python 콘솔 프로그램 1개와 GitHub 저장소 1개를 완성한다.

**1. 동작하는 퀴즈 게임**

- 프로그램 실행 시 메뉴에서 번호를 선택하면, 선택 결과에 따라 퀴즈 출제/등록/목록/점수 확인/종료 화면이 출력된다.
- 퀴즈 풀기, 퀴즈 추가, 퀴즈 목록, 점수 확인 기능이 동작한다.
- 본인이 선택한 주제의 퀴즈가 5개 이상 포함되어 있다.
- 프로그램을 종료하고 다시 실행해도 추가한 퀴즈와 최고 점수가 유지된다. (파일 저장)

**2. 코드 구조**

- 최소 2개 이상의 클래스가 정의되어 있다. (예: Quiz, QuizGame)
- 기능별로 메서드가 분리되어 있다. (입력 처리/게임 진행/저장 로직 등)
- 데이터는 프로젝트 루트의 `state.json` 에 UTF-8 인코딩으로 저장하고 불러온다.

**3. GitHub 저장소**

- 프로젝트 코드가 GitHub에 업로드되어 있다.
- 최소 10개 이상의 의미 있는 커밋이 존재한다.
- 최소 1회 이상의 브랜치 생성 및 병합( checkout , merge ) 기록이 있다.
- clone 과 pull 을 각각 1회 이상 사용한 기록이 있다.
- README.md에 아래 항목이 포함되어 있다.
    - 프로젝트 개요
    - 퀴즈 주제 선정 이유
    - 실행 방법
    - 기능 목록
    - 파일 구조
    - 데이터 파일 설명( state.json 등)

## 3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

Python 기초

- 변수가 무엇이고, 왜 사용하는지 설명할 수 있다.
- int , str , bool , list , dict 의 차이를 설명할 수 있다.
- if/elif/else 로 조건에 따라 다른 동작을 수행할 수 있다.
- for 와 while 의 차이를 설명하고 적절히 선택할 수 있다.
- 함수를 정의하고, 매개변수와 반환값을 활용할 수 있다.

클래스와 객체

- 클래스가 무엇이고, 왜 사용하는지 설명할 수 있다.
- `__init__` 메서드와 `self` 의 역할을 설명할 수 있다.
- 클래스의 속성(attribute)과 메서드(method)를 정의하고 활용할 수 있다.

파일 입출력

- 파일을 열고, 읽고, 쓰는 기본 과정을 설명할 수 있다.
- JSON 형식이 무엇이고, 왜 데이터 저장에 사용하는지 설명할 수 있다.2
- `try/except` 를 사용하여 오류를 처리할 수 있다.

Git 기초

- Git이 무엇이고 왜 필요한지 설명할 수 있다.
- `init` , `add` , `commit` , `push` , `pull` , `checkout` , `clone` 이 각각 무엇을 하는지 설명할 수 있다.
- 브랜치를 생성하고 병합할 수 있다.
- 원격 저장소를 `clone` 하고, `pull` 로 변경사항을 가져올 수 있다.

## 4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다. 구현 방법과 코드 구조는 스스로 설계한다.

**1. Git 저장소 설정**

- GitHub에 새 저장소를 생성한다.
- 로컬에 저장소를 설정한다.
- `.gitignore` , `README.md` 파일을 생성한다.
- Git: 초기 설정 완료 후 첫 번째 `commit` 과 `push` 를 수행한다.

**2. 메뉴 기능**

- 프로그램 실행 시 메뉴가 출력되어야 한다.
- 사용자가 기능을 선택할 수 있어야 한다.
- 종료 기능이 있어야 한다.
- 잘못된 입력에 대한 처리가 있어야 한다.
- Git: 메뉴 기능 완성 후 커밋한다.

**3. 공통 입력/예외 처리 기준 (최소 요구)**

- 숫자 입력이 필요한 곳에서 다음 최소 케이스를 처리해야 한다.
  - 입력 앞뒤 공백 제거 후 처리한다. (예: ` 1 `)
  - 숫자 변환 실패(예: `abc`) 시 안내 메시지 출력 후 재입력 흐름으로 복귀한다.
  - 허용 범위 밖 숫자(예: 메뉴 9, 정답 0) 입력 시 안내 메시지 출력 후 재입력 흐름으로 복귀한다.
  - 빈 입력(그냥 Enter)인 경우 안내 메시지 출력 후 재입력 흐름으로 복귀한다.
- 프로그램 실행 중 `Ctrl+C` (KeyboardInterrupt) 또는 입력 스트림 종료(EOFError)가 발생해도 “비정상 종료”하지 않도록 처리한다.
  - 안내 메시지를 출력하고, 가능한 범위에서 저장 후 안전하게 종료한다.
- 데이터 파일이 없거나 손상된 경우에도 프로그램이 실행 가능해야 한다.
  - 파일이 없으면 기본 퀴즈 데이터를 사용한다.
  - 파일이 손상되었으면 안내 메시지를 출력하고, 기본 퀴즈 데이터로 복구(또는 초기화)한다.

**4. Quiz 클래스**

- 개별 퀴즈를 표현하는 `Quiz` 클래스를 정의한다.

- 속성: 문제(question), 선택지(choices), 정답(answer)
    - 선택지는 4개를 기본으로 한다.
    - 정답은 1~4 중 하나의 번호로 관리하는 방식을 권장한다. (구현에서 일관성 유지)
- 메서드: 퀴즈 출력, 정답 확인 등 필요한 기능을 구현한다.
- Git: Quiz 클래스 작성 후 커밋한다.

**5. 기본 퀴즈 데이터**

- 본인이 선택한 주제의 퀴즈 5개 이상을 직접 작성한다.
- 각 퀴즈는 문제, 선택지(4개), 정답을 포함한다.
- Quiz 클래스의 인스턴스로 퀴즈를 생성한다.
- Git: 기본 퀴즈 데이터 작성 후 커밋한다.

**6. 퀴즈 풀기 (브랜치 활용)**

- Git: `main` 브랜치 이외의 추가 브랜치를 생성하고 작업한다.
- 저장된 퀴즈를 출제한다.
- 사용자가 정답을 입력할 수 있다.
- 정답/오답 여부를 알려준다.
- 모든 문제를 풀면 결과를 표시한다.
- 퀴즈가 없는 경우를 처리한다.
- Git: 기능 완성 후 커밋하고 `main` 브랜치로 병합한다.

**7. 퀴즈 추가**

- 새로운 퀴즈를 등록할 수 있다.

- 필요한 정보(문제, 선택지 4개, 정답 번호)를 입력받는다.
- 잘못된 입력에 대한 처리가 있어야 한다. (공통 입력/예외 처리 기준 포함)
- 추가한 퀴즈를 파일에 저장한다.
- Git: 퀴즈 추가 기능 완성 후 커밋한다.

**8. 퀴즈 목록**

- 저장된 퀴즈 목록을 확인할 수 있다.
- 퀴즈가 없는 경우를 처리한다.
- Git: 퀴즈 목록 기능 완성 후 커밋한다.

**9. 점수 확인**

- 최고 점수를 확인할 수 있다.
- 퀴즈를 풀 때마다 최고 점수와 비교하고 갱신한다.
- 최고 점수를 파일에 저장한다.
- 아직 퀴즈를 풀지 않은 경우를 처리한다.
- Git: 점수 기능 완성 후 커밋한다.

10. **QuizGame 클래스**
    * 게임 전체를 관리하는 `QuizGame` 클래스를 정의한다.
    * 속성: 퀴즈 목록, 최고 점수 등
    * 메서드: 메뉴 표시, 퀴즈 풀기, 퀴즈 추가, 목록 보기, 점수 확인, 파일 저장/불러오기 등
    * Git: 클래스 구조 정리 후 커밋한다.
11. **파일 저장/불러오기 (** **state.json** **)**
    * 퀴즈 데이터와 최고 점수를 프로젝트 루트의 `state.json` 에 저장하는 기능을 구현한다. (UTF-8 인코딩 권장)
    * JSON 파일에서 데이터를 불러오는 기능을 구현한다.

* 파일이 없는 경우(첫 실행) 기본 퀴즈 데이터를 사용한다.
* 파일이 손상되었거나 읽기/쓰기 오류가 발생한 경우를 `try/except` 로 처리한다.
* 최소 스키마 예시(데이터 형태 참고; 키 이름/구조는 일관되게 유지):
* quizzes: 퀴즈 목록
* best_score: 최고 점수(또는 최고 정답 수 등)
* Git: 파일 입출력 기능 완성 후 커밋한다.

12. **README.md 작성**
* README.md에 아래 항목이 포함되어야 한다.
* 프로젝트 개요
* 퀴즈 주제와 선정 이유
* 실행 방법
* 기능 목록
* 파일 구조
* 데이터 파일 설명( `state.json` 경로/역할/스키마)
* Git: README 작성 후 최종 푸시한다.

13. **Git 저장소 복제 실습**
* 이 단계는 `clone` 과 `pull` 명령어를 자연스럽게 경험하기 위한 실습이다. 퀴즈 게임 개발이 완료된 후 아래 절차를 순서대로 수행한다.
* 미션 수행 저장소를 `clone` 하여 별도의 로컬 디렉터리에 복제한다.
* 복제된 저장소에서 간단한 변경(예: README에 한 줄 추가)을 하고 `commit` → `push` 한다.
* 기존 로컬 작업 디렉터리에서 `pull` 로 변경사항을 가져온다.
* `pull` 받은 내용이 정상적으로 반영되었는지 확인한다.

## 5. 보너스 과제 (선택)

1. 랜덤 출제

- 퀴즈 풀기 시 문제 순서를 랜덤하게 섞는다.

- random 모듈 사용법을 스스로 학습한다.

2. 문제 수 선택

- 퀴즈 풀기 시 몇 문제를 풀지 선택할 수 있다.

3. 힌트 기능

- Quiz 클래스에 힌트 속성을 추가한다.
- 풀이 중 힌트를 볼 수 있다.
- 힌트 사용 시 점수 차감 로직을 구현한다.

4. 퀴즈 삭제 기능

- 등록된 퀴즈를 삭제할 수 있다.
- 삭제 후 파일에 반영한다.

5. 점수 기록 히스토리

- 최고 점수뿐 아니라 모든 게임 기록을 저장한다.
- 날짜/시간, 푼 문제 수, 점수를 기록한다.

---

개발환경

## 6. 개발 환경

- Python 3.10 이상을 사용해야 한다.

- 외부 라이브러리 없이 기본 문법만 사용해야 한다. (표준 라이브러리 사용 가능)

---

제약조건

## 7. 제약 사항

- 데이터 저장 규칙
    - 데이터 파일은 프로젝트 루트의  state.json 을 기본으로 한다.
    - 파일 인코딩은 UTF-8을 권장한다.
- 코드 구조
    - 모든 코드를 한 함수에 작성하지 않고, 기능별로 함수를 분리해야 한다.
    - 최소 2개 이상의 클래스로 역할을 분리해야 한다.
- Git 워크플로우
    - 최소 10개 이상의 의미 있는 커밋이 있어야 한다.
        - 기능 단위 커밋(메뉴/Quiz/플레이/추가/저장/README 등) + 커밋 메시지에 변경 요약 포함
    - 형식적인 커밋 메시지를 피하고, 아래처럼 작업 내용을 드러내는 형식을 권장한다.
        - Feat: 퀴즈 출제 기능 구현
        - Fix: 점수 계산 오류 수정
        - Docs: README 실행 방법 추가

        - Refactor: QuizGame 책임 분리
    - Git 기초 명령어 7종( `init` , `add` , `commit` , `push` , `pull` , `checkout` , `clone` )을 각각 한 번 이상 사용해야 한다.
- 제출물
    - GitHub 저장소 URL
    - 개발 환경 설정 스크린샷(예: VSCode, Python 버전, Git 설정)
    - 프로그램 실행 결과 스크린샷(퀴즈 추가, 목록, 플레이, 점수)
    - `git log --oneline --graph` 결과 스크린샷

---

Test Case

## 8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 달라도 된다.

- 메뉴 화면(예시)

```
CSS
==========================================
      🎯 나만의 퀴즈 게임 🎯
==========================================
1. 퀴즈 풀기
2. 퀴즈 추가
3. 퀴즈 목록
4. 점수 확인
```

```
5. 종료
========================================
선택:
```

- 퀴즈 풀기(예시)

```
CSS
선택: 1

📝 퀴즈를 시작합니다! (총 5문제)

----------------------------------------
[문제 1]
마블 시네마틱 유니버스에서 타노스가 모은 인피니티 스톤의 개수는?

1. 4개
2. 5개
3. 6개
4. 7개

정답 입력: 3
✅ 정답입니다!

----------------------------------------
... (중략) ...

========================================
```

```
🏆 결과: 5문제 중 4문제 정답! (80점)
🎉 새로운 최고 점수입니다!
==========================================
```

- 퀴즈 추가(예시)

```
CSS
선택: 2

📌 새로운 퀴즈를 추가합니다.

문제를 입력하세요: 영화 '기생충'의 감독은?
선택지 1: 박찬욱
선택지 2: 봉준호
선택지 3: 김기덕
선택지 4: 이창동
정답 번호 (1-4): 2

✅ 퀴즈가 추가되었습니다!
```

- 퀴즈 목록(예시)

```
CSS
선택: 3

📋 등록된 퀴즈 목록 (총 6개)
```

```
---------------------------------------
[1] 마블 시네마틱 유니버스에서 타노스가 모은 인피니티 스톤의 개수는?
[2] 영화 '인터스텔라'에서 주인공이 방문하지 않은 행성은?
...
---------------------------------------
```

- 점수 확인(예시)

```
선택: 4

🏆 최고 점수: 80점 (5문제 중 4문제 정답)
```

- 잘못된 입력 처리(예시)

```
선택: abc
⚠️ 잘못된 입력입니다. 1-5 사이의 숫자를 입력하세요.

선택: 9
⚠️ 잘못된 입력입니다. 1-5 사이의 숫자를 입력하세요.
```

- 프로그램 재시작 후 데이터 유지 확인(예시)

```
==========================================
      🎯 나만의 퀴즈 게임 🎯
==========================================
📂 저장된 데이터를 불러왔습니다. (퀴즈 6개, 최고점수 80점)
==========================================
1. 퀴즈 풀기
2. 퀴즈 추가
...
```

- state.json 예시 (데이터 형태 참고)

```
{
  "quizzes": [
    {
      "question": "Python의 창시자는?",
      "choices": ["Guido", "Linus", "Bjarne", "James"],
      "answer": 1
    }
  ],
  "best_score": 3
}
```

- README 제출 체크리스트

    - 프로젝트 개요
    - 퀴즈 주제 선정 이유
    - 실행 방법 (예: `python main.py`)
    - 기능 목록(퀴즈 풀기/추가/목록/점수)
    - 파일 구조
    - 데이터 파일 설명(`state.json` 경로/역할/필드 구조)
- 실행 화면 스크린샷 구성 예시(학습자 캡처)
    - `docs/screenshots/menu.png`
    - `docs/screenshots/play.png`
    - `docs/screenshots/add_quiz.png`
    - `docs/screenshots/score.png`